

Core Java – Chapter 10

java.util Package & The Collection Framework

1. What is the java.util Package?

The **java.util** package is one of the most important built-in packages in Java. It provides utility classes that make Java programming easier and more powerful, helping with:

- Data storage (Collections)
- Working with dates
- Working with random numbers
- Reading input
- Handling arrays
- Using scanners, lists, maps and sets

Why do we need it? Core Java by itself does not provide built-in support for tasks such as storing multiple objects easily, sorting data, reading user input, working with dates/times, or generating random numbers — so Java bundles ready-made utility classes inside java.util to handle these.

Common Classes in java.util

Class Name	Purpose
Scanner	To take input from the user
ArrayList	Growable array
HashMap	Key–value pair map
HashSet	Set of unique values
LinkedList	List built from linked nodes
Collections	Utility methods (sort, reverse, ...)
Date	Work with dates
Random	Generate random values
Arrays	Utility methods for arrays

2. What is the Collections Framework?

The Java Collections Framework is a set of classes and interfaces that help store, manage and process groups of objects efficiently. It provides:

- Resizable data structures
- Built-in algorithms (sorting, searching, etc.)
- Easy data processing
- Ready-made classes like ArrayList, HashSet and HashMap

Why Collections instead of Arrays?

Arrays	Collections
Fixed size	Grow / shrink dynamically
Can store only the same type	Can store any reference type
No built-in algorithms	Has sorting, searching, etc.
Manual operations	Ready-made utility methods

Collection Framework Hierarchy

Iterable

↑

Collection

↑ ————— ↑ ————— ↑

List

Set

Queue

- **List** (java.util.List) – ordered, duplicates allowed → ArrayList, LinkedList, Vector, Stack
- **Set** (java.util.Set) – no duplicates → HashSet, LinkedHashSet, TreeSet (SortedSet)
- **Queue** (java.util.Queue) – FIFO behaviour → PriorityQueue, Deque

Note: Map is NOT part of the Collection interface hierarchy. Map (with implementations HashMap, Hashtable, etc.) is a separate, parallel interface in the Java Collections Framework.

3. Collection vs. Collections

Collection (interface) → represents a group of objects. **Collections** (class, in java.util.Collections) → a utility/helper toolbox of static methods used to operate on collection objects like List, Set and Queue (e.g. sort, reverse, shuffle, find max/min, make thread-safe or read-only).

Feature	Collection	Collections
Type	Interface	Utility (helper) class
Package	java.util.Collection	java.util.Collections
Purpose	Represent a group of objects	Provide utility methods for collections
Contains	Abstract methods (add, remove, size)	Static methods (sort, reverse, shuffle)
Used for	Creating collection objects (List, Set, Queue)	Performing operations on collection objects
Inheritance	Extended by List, Set, Queue	Cannot be extended; final class
Object creation	Cannot instantiate directly (interface)	Cannot create an object; methods are static
Example	Collection<String> c = new ArrayList<>();	Collections.sort(list);

Part of hierarchy?	Yes – core part of the Collection Framework	No – sits outside as a helper class
--------------------	---	-------------------------------------

Key uses of the Collections class: Collections.sort(list) · Collections.reverse(list) · Collections.shuffle(list) · Collections.max(list) / Collections.min(list) · Collections.synchronizedList(list) (thread-safe) · Collections.unmodifiableList(list) (read-only).

Typical uses of the Collection Framework: storing records (students, employees), handling data from databases, managing lists in real apps (e.g. shopping carts), and processing data in algorithms.

4. The List Interface

List is a subinterface of Collection representing an ordered collection where duplicates are allowed, elements are accessed by a 0-based index, and insertion order is maintained.

```
List<DataType> listName = new ArrayList<>();
```

Implementations of List

Class	Key Points
ArrayList	Dynamic array that grows automatically; allows duplicates; maintains insertion order; fast index-based access (get); slower insert/delete in the middle.
LinkedList	Based on a doubly-linked list (each node stores data + pointers to next and previous nodes); best for insertion/deletion; slower for random access (get()); also implements Queue (offer/poll).
Vector	Dynamic array similar to ArrayList but synchronized (thread-safe); slower than ArrayList; maintains order; allows duplicates; grows by doubling size; legacy class.
Stack	Subclass of Vector; follows LIFO (Last In, First Out); supports push(), pop(), peek(); useful for undo, recursion, expression evaluation; thread-safe (inherits Vector) but slower; legacy — modern alternative is Deque.

5. The Set Interface

Set is a subinterface of Collection that represents a group of unique elements.

- Does NOT allow duplicates.
- Does NOT maintain insertion order (except LinkedHashSet).
- At most one null element (TreeSet does NOT allow null).

```
Set<Type> set = new HashSet<>();
```

Implementations of Set

Class	Key Points
HashSet	Stores elements in random/unordered fashion; fastest Set implementation; allows one null value.
LinkedHashSet	Keeps elements in the order they were inserted; slightly slower than HashSet.

TreeSet	Stores elements in sorted (ascending) order; does NOT allow null; implemented internally using a Red-Black Tree.
---------	--

Common Set methods (available on all Set types): add(E e), remove(Object o), contains(Object o), size(), clear(), isEmpty(), iterator().

6. The Map Interface

Map is part of the Java Collection Framework, but it is **NOT** a subinterface of Collection. It stores data as key–value pairs.

- Keys are unique (no duplicates).
- Values can be duplicated.
- One key maps to exactly one value.
- Fast lookup using keys.

```
Map<KeyType, ValueType> map = new HashMap<>();
```

Implementations of Map

Class	Key Points
HashMap	Fast; no order maintained; allows one null key.
LinkedHashMap	Maintains insertion order.
TreeMap	Sorted by keys; no null key allowed.
Hashtable	Synchronized (thread-safe); no null key or value allowed.

Common ways to work with a Map: students.get(key) to read a value, and loop via keySet() (keys), values() (values), or entrySet() (key–value pairs together).

7. The Queue Interface

Queue is a subinterface of Collection representing a collection designed for holding elements before processing, typically in FIFO (First In, First Out) order.

- FIFO order — the first element added is the first removed.
- Duplicates are allowed.
- Null handling varies: LinkedList allows null, but ArrayDeque and most others do NOT.

Implementations of Queue

Class	Key Points
LinkedList	Implements Queue; allows null.
PriorityQueue	Elements are ordered by priority (sorted order), not insertion order; no null allowed.
ArrayDeque	Fast double-ended queue; no null allowed; recommended modern replacement for Stack.

Useful Queue methods: `add()` (throws exception if it fails), `offer()` (returns false if it fails), `poll()` (removes head, returns null if empty), `remove()` (removes head, throws exception if empty), `peek()` / `element()` (view head without removing).

Where Queues are used: task scheduling, printer queues, CPU job scheduling, Breadth-First Search (BFS) in graphs, and message handling.

8. Difference Between List, Set and Map

Feature	List	Set	Map
Stores	Values	Unique values	Key–value pairs
Order	Ordered	Unordered (except LinkedHashSet / TreeSet)	Keys unordered (depends on type)
Duplicates	Allowed	Not allowed	Keys must be unique
Access	Via index	No index	Via key
Examples	ArrayList, LinkedList	HashSet, TreeSet	HashMap, TreeMap

9. Cursors in Java (Ways to Iterate Over Collections)

Java provides three types of cursors for looping through collections: **Iterator**, **ListIterator**, and **Enumeration**. These are used to traverse elements of collections like List, Set, Vector, etc.

Iterator

- Universal cursor — works with any collection (List, Set, Queue).
- Moves forward only (one direction).
- Allows `remove()` during iteration.
- The modern replacement for Enumeration.
- Key methods: `hasNext()`, `next()`, `remove()`.

ListIterator

- Works only with List (ArrayList, LinkedList).
- Supports two-way traversal — can move forward and backward.
- Can `add()`, `remove()`, and `set()` elements during iteration.
- More powerful than a plain Iterator.
- Key methods: `hasNext()/next()`, `hasPrevious()/previous()`, `add()`, `remove()`, `set()`.

Enumeration

- Legacy cursor used only with old classes: Vector, Stack, Hashtable.
- Moves forward only and cannot remove elements.
- The oldest cursor type, predating Iterator.
- Key methods: `hasMoreElements()`, `nextElement()`.

Cursor Comparison

Cursor	Works With	Direction	Can Modify?
Iterator	Any Collection (List, Set, Queue)	Forward only	Can remove()
ListIterator	List only (ArrayList, LinkedList)	Forward and backward	Can add(), remove(), set()
Enumeration	Legacy classes (Vector, Stack, Hashtable)	Forward only	Cannot modify

10. Quick-Reference: Differences Between Implementation Classes

Interface	Class	Ordered?	Duplicates?	Notes
List	ArrayList	Yes (insertion)	Yes	Fast access, slow middle insert/delete
List	LinkedList	Yes (insertion)	Yes	Fast insert/delete, slow random access
List	Vector	Yes (insertion)	Yes	Synchronized / thread-safe, legacy
List	Stack	LIFO	Yes	Subclass of Vector
Set	HashSet	No	No	Fastest Set, allows 1 null
Set	LinkedHashSet	Yes (insertion)	No	Slightly slower than HashSet
Set	TreeSet	Yes (sorted)	No	No null, uses Red-Black Tree
Map	HashMap	No	Unique keys	Allows 1 null key
Map	LinkedHashMap	Yes (insertion)	Unique keys	-
Map	TreeMap	Yes (sorted by key)	Unique keys	No null key
Map	Hashtable	No	Unique keys	Synchronized, no null key/value
Queue	PriorityQueue	Sorted (priority)	Yes	No null allowed
Queue	ArrayDeque	Insertion (double-ended)	Yes	No null; replaces Stack

End of Chapter 10 – java.util Package & Collection Framework